

Міністерство освіти і науки України
Національний Університет “Львівська Політехніка”



Лабораторна робота
з дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав:
студент гр. ІХ-22
Пристайко М.

Прийняв:
Гордійчук-Бублівська О.В

Лабораторна робота №1

Тема: Застосування принципів SOLID об'єктно-орієнтованого програмування при проєктуванні програмних систем у сфері телекомунікацій.

Мета: Ознайомитися з принципами SOLID. Навчитися застосовувати їх під час проєктування класів на Python. Формувати навички створення гнучкого, масштабованого та підтримуваного коду.

1. Single Responsibility Principle (SRP)

1.1

Є клас CallReport, який формує звіт про дзвінки та зберігає його у файл.

Розділити відповідальності відповідно до SRP.

class Call:

```
def __init__(self, number: str, minutes: int, rate: float):
    self.number = number
    self.minutes = minutes
    self.rate = rate
```

class CallCostCalculator:

```
def calculate(self, call: Call) -> float:
    return call.minutes * call.rate
```

class CallReportBuilder:

```
def build_report(self, calls: list) -> str:
    calculator = CallCostCalculator()
    lines = []
    total_sum = 0
    for call in calls:
        cost = calculator.calculate(call)
        total_sum += cost
        lines.append(f'Number: {call.number}, Duration: {call.minutes} min, Cost:
{cost:.2f} UAH")
    lines.append(f'\nTotal cost: {total_sum:.2f} UAH")
    return "\n".join(lines)
```

class ReportFileWriter:

```
def write(self, filename: str, report: str):
    with open(filename, "w", encoding="utf-8") as file:
        file.write(report)
```

if __name__ == "__main__":

```
    call_list = [
        Call("0501234567", 5, 2.0),
        Call("0679876543", 8, 2.0)
    ]
    report = CallReportBuilder().build_report(call_list)
```

```
ReportFileWriter().write("calls_report.txt", report)
print("Report successfully created.")
```

Результат:

Report successfully created.

1.2

Клас Subscriber:

зберігає дані абонента

відправляє SMS

розраховує баланс

Перепроєктувати систему так, щоб кожен клас мав одну відповідальність.

```
from abc import ABC, abstractmethod
```

```
class Tariff(ABC):
    @abstractmethod
    def calculate_cost(self, usage: float) -> float:
        pass
```

```
class BasicTariff(Tariff):
    def calculate_cost(self, minutes: float) -> float:
        return minutes * 1.5
```

```
class PremiumTariff(Tariff):
    def calculate_cost(self, minutes: float) -> float:
        return minutes * 3.0 + 2
```

```
class BillingSystem:
    def compute_cost(self, tariff: Tariff, usage: float) -> float:
        return tariff.calculate_cost(usage)
```

```
if __name__ == "__main__":
    billing = BillingSystem()
    basic_plan = BasicTariff()
    premium_plan = PremiumTariff()
    print("Basic call 10 min:", billing.compute_cost(basic_plan, 10), "UAH")
    print("Premium call 5 min:", billing.compute_cost(premium_plan, 5), "UAH")
```

Результат:

Basic call 10 min: 15.0 UAH

Premium call 5 min: 17.0 UAH

2. Open/Closed Principle (OCP)

2.1

Реалізувати систему тарифікації дзвінків з базовим тарифом.

Додати новий тариф без зміни існуючого коду.

```
from abc import ABC, abstractmethod
```

```
class Tariff(ABC):
    @abstractmethod
    def calculate_cost(self, usage: float) -> float:
        pass
```

```
class VoiceTariff(Tariff):
    def calculate_cost(self, minutes: float) -> float:
        return minutes * 2.0
```

```
class DataTariff(Tariff):
    def calculate_cost(self, megabytes: float) -> float:
        return megabytes * 0.05
```

```
class RoamingTariff(Tariff):
    def calculate_cost(self, minutes: float) -> float:
        return minutes * 10.0 + 5.0
```

```
class BillingSystem:
    def compute_cost(self, tariff: Tariff, usage: float) -> float:
        return tariff.calculate_cost(usage)
```

```
if __name__ == "__main__":
    billing = BillingSystem()
    voice_plan = VoiceTariff()
    data_plan = DataTariff()
    roaming_plan = RoamingTariff()
    print("Voice call 15 min:", billing.compute_cost(voice_plan,
15), "UAH")
    print("Internet usage 358 MB:", billing.compute_cost(data_plan,
358), "UAH")
    print("Roaming call 2.9 min:",
billing.compute_cost(roaming_plan, 2.9), "UAH")
```

Результат:

Voice call 15 min: 30.0 UAH

Internet usage 358 MB: 17.900000000000002 UAH

Roaming call 2.9 min: 34.0 UAH

2.2

Система підтримує тарифи: VoiceTariff, DataTariff.

Розширити систему тарифом RoamingTariff, не змінюючи логіку розрахунку вартості

```
from abc import ABC, abstractmethod
```

```
class NetworkConnection(ABC):
```

```
    @abstractmethod
    def connect(self):
        pass
```

```
    @abstractmethod
    def send_data(self, data: str):
        pass
```

```
class LTEConnection(NetworkConnection):
```

```
    def connect(self):
        print("LTE: Підключення встановлено через стільникову вежу.")

    def send_data(self, data: str):
        print(f"LTE: Дані '{data}' надіслано.")
```

```
class WiFiConnection(NetworkConnection):
```

```
    def connect(self):
        print("WiFi: Підключення встановлено.")

    def send_data(self, data: str):
        print(f"WiFi: Дані '{data}' надіслано.")
```

```
def transmit_report(connection: NetworkConnection):
```

```
    print(f"\n--- Робота з {type(connection).__name__} ---")
    connection.connect()
    connection.send_data("Звіт за квартал")
```

```
if __name__ == "__main__":  
    connections = [  
        LTEConnection(),  
        WiFiConnection()  
    ]  
    for conn in connections:  
        transmit_report(conn)
```

Результат:

--- Робота з LTEConnection ---

LTE: Підключення встановлено через стільникову вежу.

LTE: Дані 'Звіт за квартал' надіслано.

--- Робота з WiFiConnection ---

WiFi: Підключення встановлено.

WiFi: Дані 'Звіт за квартал' надіслано.

3. Liskov Substitution Principle (LSP)

3.1

Є базовий клас `NetworkConnection`.

Реалізувати підкласи `LTEConnection`, `WiFiConnection`, які можна взаємозамінно використовувати.

```
from abc import ABC, abstractmethod
```

```
class NetworkConnection(ABC):
```

```
    @abstractmethod
```

```
    def connect(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def send_data(self, data: str):
```

```
        pass
```

```
class LTEConnection(NetworkConnection):
```

```
    def connect(self):
```

```
        print("LTE: Підключення встановлено через стільникову вежу.")
```

```
    def send_data(self, data: str):
```

```
        print(f"LTE: Дані '{data}' надіслано.")
```

```
class WiFiConnection(NetworkConnection):
```

```
    def connect(self):
```

```
        print("WiFi: Підключення встановлено.")
```

```
    def send_data(self, data: str):
```

```
        print(f"WiFi: Дані '{data}' надіслано.")
```

```
class SatelliteConnection(ABC):
```

```
    @abstractmethod
```

```
    def establish_satellite_link(self):
```

```
        pass
```

```
class StarlinkConnection(SatelliteConnection):
    def establish_satellite_link(self):
        print("Starlink: Пошук супутників на орбіті...")

    def connect(self):
        self.establish_satellite_link()
        print("Starlink: Зв'язок встановлено через супутник.")

    def send_data(self, data: str):
        print(f"Starlink: Дані '{data}' надіслано через супутник.")

def transmit_report(connection):
    print(f"\n--- Робота з {type(connection).__name__} ---")
    if isinstance(connection, NetworkConnection):
        connection.connect()
        connection.send_data("Звіт за квартал")
    elif isinstance(connection, SatelliteConnection):
        connection.connect()
        connection.send_data("Звіт за квартал")
    else:
        print("Невідомий тип з'єднання")

if __name__ == "__main__":
    connections = [
        LTEConnection(),
        WiFiConnection(),
        StarlinkConnection()
    ]
    for conn in connections:
        transmit_report(conn)
```


Результат:

--- Робота з LTEConnection ---

LTE: Підключення встановлено через стільникову вежу.

LTE: Дані 'Звіт за квартал' надіслано.

--- Робота з WiFiConnection ---

WiFi: Підключення встановлено.

WiFi: Дані 'Звіт за квартал' надіслано.

--- Робота з StarlinkConnection ---

Starlink: Пошук супутників на орбіті...

Starlink: Зв'язок встановлено через супутник.

Starlink: Дані 'Звіт за квартал' надіслано через супутник.

3.2 Виправити створену ієрархію, де клас `SatelliteConnection` порушує очікувану поведінку базового класу (супутник не може працювати як звичайне з'єднання).

```
from abc import ABC, abstractmethod
```

```
class Callable(ABC):
```

```
    @abstractmethod
```

```
    def make_call(self, number: str):
```

```
        pass
```

```
class SMSCapable(ABC):
```

```
    @abstractmethod
```

```
    def send_sms(self, number: str, message: str):
```

```
        pass
```

```
class NetworkConnectable(ABC):
```

```
    @abstractmethod
```

```
    def connect_to_network(self):
```

```
        pass
```

```
class Smartphone(Callable, SMSCapable, NetworkConnectable):
```

```
    def make_call(self, number: str):
```

```
        print(f"Smartphone: Дзвінок на номер {number} здійснено.")
```

```
    def send_sms(self, number: str, message: str):
```

```
        print(f"Smartphone: SMS на {number} - '{message}' надіслано.")
```

```
    def connect_to_network(self):
```

```
        print("Smartphone: Підключення до мобільної мережі  
встановлено.")
```

```
if __name__ == "__main__":
```

```
    phone = Smartphone()
```

```
phone.connect_to_network()
phone.make_call("0501234567")
phone.send_sms("0501234567", "Привіт!")
```

Результат:

Smartphone: Підключення до мобільної мережі встановлено.

Smartphone: Дзвінок на номер 0501234567 здійснено.

Smartphone: SMS на 0501234567 - 'Привіт!' надіслано.

```
from abc import ABC, abstractmethod
```

```
class DataTransferable(ABC):
```

```
    @abstractmethod
```

```
    def send_data(self, data: str):
```

```
        pass
```

```
class IoTDevice(DataTransferable):
```

```
    def send_data(self, data: str):
```

```
        print(f'IoT Device: Дані '{data}' передано на сервер.")
```

```
if __name__ == "__main__":
```

```
    sensor = IoTDevice()
```

```
    sensor.send_data("Температура: 22°C")
```

```
    sensor.send_data("Вологість: 55%")
```

Результат:

IoT Device: Дані 'Температура: 22°C' передано на сервер.

IoT Device: Дані 'Вологість: 55%' передано на сервер.

4. Interface Segregation Principle (ISP)

4.1

Інтерфейс TelecomDevice має методи:

- make_call()
- send_sms()
- connect_to_network()

Розділити інтерфейс відповідно до ISP.

```
class FileLogger:
    def log(self, message: str):
        print(f"[FileLogger] Запис у файл: {message}")

class NetworkMonitor:
    def __init__(self):
        self.logger = FileLogger()

    def check_connection(self):
        status = "Мережа працює стабільно"
        self.logger.log(status)

if __name__ == "__main__":
    monitor = NetworkMonitor()
    monitor.check_connection()
```

Результат:

[FileLogger] Запис у файл: Мережа працює стабільно

5.2

Реалізувати можливість підключення:

- FileLogger
- ServerLogger
- ConsoleLogger

(без змін у коді класу NetworkMonitor).

```
from abc import ABC, abstractmethod
```

```
class ILogger(ABC):
    @abstractmethod
    def log(self, message: str):
        pass
```

```
class FileLogger(ILogger):
    def log(self, message: str):
        print(f"[FileLogger] Запис у файл: {message}")
```

```
class ServerLogger(ILogger):
    def log(self, message: str):
        print(f"[ServerLogger] Відправка на сервер: {message}")
```

```
class ConsoleLogger(ILogger):
    def log(self, message: str):
        print(f"[ConsoleLogger] Відображення: {message}")
```

```
class NetworkMonitor:
    def __init__(self, logger: ILogger):
        self.logger = logger

    def check_connection(self):
        status = "Мережа працює стабільно"
        self.logger.log(status)
```

```
if __name__ == "__main__":
    monitor_console = NetworkMonitor(ConsoleLogger())
    monitor_console.check_connection()
```

```
    monitor_file = NetworkMonitor(FileLogger())
    monitor_file.check_connection()
```

```
    monitor_server = NetworkMonitor(ServerLogger())
    monitor_server.check_connection()
```

Результат:

[ConsoleLogger] Відображення: Мережа працює стабільно

[FileLogger] Запис у файл: Мережа працює стабільно

[ServerLogger] Відправка на сервер: Мережа працює стабільно

Висновок:

У ході виконання даної лабораторної роботи я опанував принципи SOLID та дослідив їх практичне використання під час розробки програмного забезпечення для телекомунікаційних систем. Отримані знання дозволили мені навчитися грамотно розподіляти обов'язки між класами, будувати гнучку та легко розширювану архітектуру коду, а також використовувати абстрактні інтерфейси для покращення читабельності, підтримки та тестування програм.